

Quick Sort

↓ pivot

(50) 70 60 90 40 80 10 20 30 ∞
i ————— j

(50) 30 60 90 40 80 10 20 70 ∞
i ————— j

(50) 30 20 90 40 80 10 60 70 ∞
i ————— j

(50) 30 20 10 40 80 90 60 70 ∞
|————— j i

(40 30 20 10) (50) (80 90 60 70 ∞)

j
partitioning
position

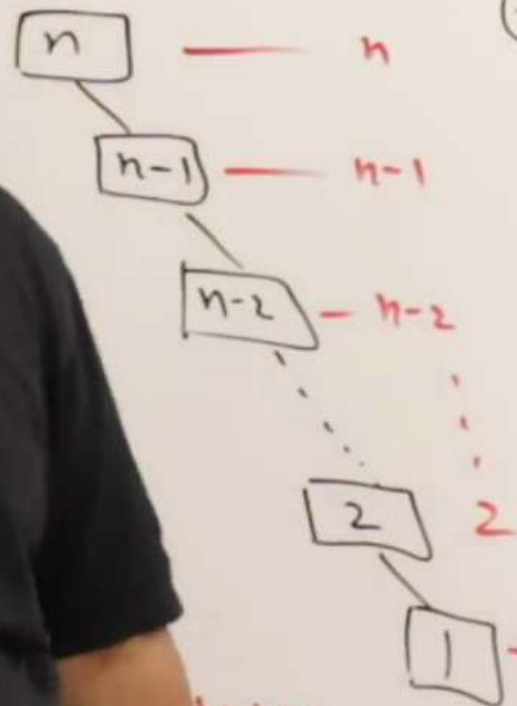
Quick Sort

<u>x</u>	1	20	30	40	50	∞	n
	j	i					
<u>x</u>		20	30	40	50	∞	n-1
		j	i				

		30	40	50	∞	n-2
		j	i			

		40	50	∞	2
		j	i		

So ∞ 1



$$1+2+3+\dots+n = \frac{n(n+1)}{2} = O(n^2)$$

```
int Partition(int A[], int l, int h)
```

```
{
    int Pivot = A[l];
    int i = l, j = h;
```

```
do
```

```
do { i++; while (A[i] <= pivot);
```

```
do { j--; while (A[j] > pivot);
```

```
if (i < j)
```

```
    swap(A[i], A[j]);
```

```
while (i < j);
```

```
swap(A[l], A[j]);
```

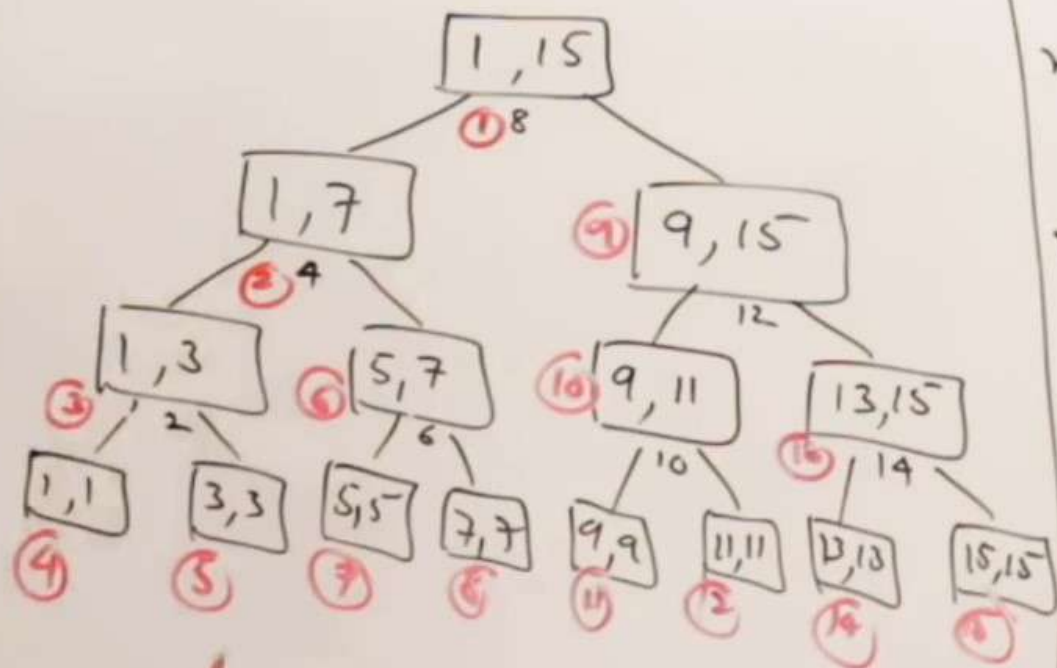
```
return j;
```

```
}
```

Quick Sort

$$\log_2 n$$

$$\frac{16}{2} = \frac{8}{2} = \frac{4}{2} = \frac{2}{2} = 1$$



$$O(n \log n)$$

```
int Partition(int A[], int l, int h)
{
    int Pivot = A[l];
    int i = l, j = h;
```

```
do
```

```
do { i++; } while (A[i] <= pivot);
```

```
do { j--; } while (A[j] > pivot);
```

```
if (i < j)
```

```
    swap(A[i], A[j]);
```

```
} while (i < j);
```

```
swap(A[l], A[j]);
```

```
return j;
```

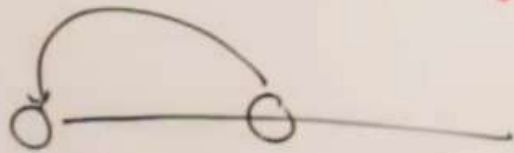
$n \log n$

Quick Sort

Best Case $\rightarrow$ if partitioning is in middle
Time $- O(n \log n)$

Worst case $\rightarrow$ if partitioning is on any end
Time $- O(n^2)$ (already sorted)

Avg. Case Time $\rightarrow O(n \log n)$



Best case $\rightarrow$ sorted list $O(n \log n)$

Worst case $\rightarrow$ partitioning at any end $O(n^2)$ }

```
int Partition(int A[], int l, int h)
{
    int Pivot = A[l];
    int i = l, j = h;
```

```
do
```

```
{
    do { i++; } while (A[i] <= Pivot);
```

```
    do { j--; } while (A[j] > Pivot);
```

```
    if (i < j)
```

```
        swap(A[i], A[j]);
```

```
    } while (i < j);
```

```
    swap(A[l], A[j]);
```

```
    return j;
```